

Inheritance in Java

Definition

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows one class (child class) to acquire the properties and methods of another class (parent class). This promotes code reuse and establishes a relationship between classes.

In Java, inheritance is implemented using the `extends` keyword.

Key Concepts of Inheritance

- Parent Class (Super Class):**
The class whose properties and methods are inherited by another class.
 - Child Class (Sub Class):**
The class that inherits the properties and methods of the parent class.
 - `extends` Keyword:**
Used to establish the parent-child relationship between classes.
 - Overriding Methods:**
A child class can redefine methods from the parent class to provide its specific implementation.
-

Types of Inheritance in Java

- Single Inheritance**
One class inherits from another class.
Example: `Child extends Parent`
 - Multilevel Inheritance**
A class inherits from another class, and that class inherits from a third class.
Example: `GrandChild extends Child extends Parent`
 - Hierarchical Inheritance**
Multiple classes inherit from a single parent class.
Example: `Child1 extends Parent, Child2 extends Parent`
 - Multiple Inheritance (via Interfaces only)**
Java does not support multiple inheritance with classes to avoid ambiguity. However, it supports multiple inheritance using interfaces.
-

Advantages of Inheritance

- Code Reusability:**
Common functionality is defined in the parent class, and child classes reuse it.
- Extensibility:**
New functionality can be added in child classes without modifying the parent class.

OUR TRENDING COURSES

- Java Fullstack Development
- Software Testing (Java-Selenium)
- UI/UX Development
- Cloud Computing



+91-9130502135

+91-8484831616

TESTING SHASTRA

WE ARE PUNE'S BEST IT TRAINING INSTITUTE

3. Polymorphism: Enables dynamic method invocation.

Code Example: Parent and Child Classes

Parent Class: Dukaandaar

```
// Parent class
class Dukaandaar {
    // Method representing the shop
    public void dukaan() {
        System.out.println("Dukaandaar: Dukaan khuli hai, saman le jao!");
    }
}
```

Child Class: Beta

```
// Child class inheriting from Dukaandaar
class Beta extends Dukaandaar {
    // Child's unique method
    public void betaKaKaam() {
        System.out.println("Beta: Main dukaan chala raha hoon aaj!");
    }
}
```

Main Class: Demonstrating Inheritance

```
// Main class to demonstrate inheritance
public class InheritanceDemo {
    public static void main(String[] args) {
        // Create an object of Beta (child class)
        Beta beta = new Beta();

        // Call the inherited method from Dukaandaar
        beta.dukaan();
    }
}
```

```
        // Call the child class method
        beta.betaKaKaam();
    }
}
```

Output

Dukaandaar: Dukaan khuli hai, saman le jao!
Beta: Main dukaan chala raha hoon aaj!

Overriding Example

The child class can override the `dukaan()` method to provide its own implementation.

```
class Beta extends Dukaandaar {
    @Override
    public void dukaan() {
        System.out.println("Beta: Aaj main dukaan sambhal raha hoon!");
    }
}
```

Output

Beta: Aaj main dukaan sambhal raha hoon!

Key Points

1. The parent class constructor is always called before the child class constructor.
 2. Java supports single and multilevel inheritance with classes but not multiple inheritance.
 3. Child classes can access parent class methods and variables unless they are private.
-

Interview Questions and Answers

Q1: What is inheritance in Java? Why is it used?

A: Inheritance is a mechanism where one class acquires the properties and methods of another. It is used for code reuse, extensibility, and to implement polymorphism.

Q2: What are the different types of inheritance in Java?

A: Java supports:

1. Single inheritance
2. Multilevel inheritance

3. Hierarchical inheritance
 4. Multiple inheritance (via interfaces)
-

Q3: Why doesn't Java support multiple inheritance with classes?

A: Java avoids multiple inheritance with classes to prevent ambiguity caused by the **Diamond Problem** (when two parent classes have methods with the same name, it becomes unclear which method the child class should inherit).

Q4: Can a child class access private members of the parent class?

A: No, private members are not accessible directly by the child class. However, they can be accessed through public or protected methods of the parent class.

Q5: What is the difference between `is-a` and `has-a` relationships?

A:

- **`is-a` Relationship:** Achieved through inheritance (e.g., Beta is a Dukaandaar).
 - **`has-a` Relationship:** Achieved through composition (e.g., Dukaandaar has a Shop object).
-

Q6: Can you inherit constructors in Java?

A: No, constructors are not inherited. However, the child class can call the parent class constructor using `super()`.
